

IL355 — PROGRAMACIÓN DE SISTEMAS AVANZADOS · FASE 1

PIPELINE **DISTRIBUIDO**

IOT / EDGE

Integrantes del equipo

Pérez Hernández Zacarias

Alonso Reyes Ángel Gabriel

OBJETIVO Y ALCANCE



Pipeline IoT/Edge

Implementar un sistema distribuido con tolerancia a fallos para asegurar la continuidad operativa.



Seguridad WireGuard

Establecer conectividad cifrada y segura entre todos los nodos de la red.



Despliegue Docker

Contenerización de componentes: sensor, edge y coordinator para facilitar el despliegue.



Validación de Red

Pruebas de comunicación, heartbeat y latencia utilizando herramientas como tc netem.

ARQUITECTURA GENERAL



SENSOR

Primer eslabón del pipeline.
Encargado de la captura de
datos y envío inicial hacia la capa
intermedia.



EDGE

Nodo de procesamiento local.
Filtrar y procesa información
antes de la consolidación
centralizada.



COORDINATOR

Punto central de gestión.
Coordina la lógica del sistema y
el almacenamiento final de datos
procesados.

Infraestructura: Servicios en red Docker bridge. **Desarrollo:** Workspace Rust con crates modulares por responsabilidad.

¿QUÉ ES TC?



Herramienta de Linux (iproute2)

Traffic Control gestiona el comportamiento del tráfico de red directamente en el kernel de Linux.



Gestión de Paquetes

Controla cómo se envían los paquetes a través de las interfaces de red del sistema.



Optimización (QoS)

Permite aplicar políticas de Calidad de Servicio, priorizar flujos y limitar el ancho de banda.



Control de Red Avanzado

Es fundamental para el diseño de infraestructuras de red robustas y servicios distribuidos.

¿QUÉ ES NETEM?



Network Emulator

Disciplina de cola del subsistema de tráfico de Linux.



Simulación Controlada

Reproduce condiciones de red reales de forma controlada para entornos de desarrollo.



Inyección de Fallos

Introduce latencia, jitter, pérdida, duplicación y corrupción de paquetes.



Pruebas de Estrés

Degrada el tráfico intencionalmente para validar la robustez de los protocolos.

RED VPN WIREGUARD

Topología Hub & Spoke

Estructura de red centralizada para comunicación eficiente entre nodos.



Nodos de Red

Hub Central: 10.10.10.1

Spokes: 10.10.10.2, 10.10.10.3



Seguridad y Acceso

Intercambio de claves públicas y definición de AllowedIPs para validación de tráfico.



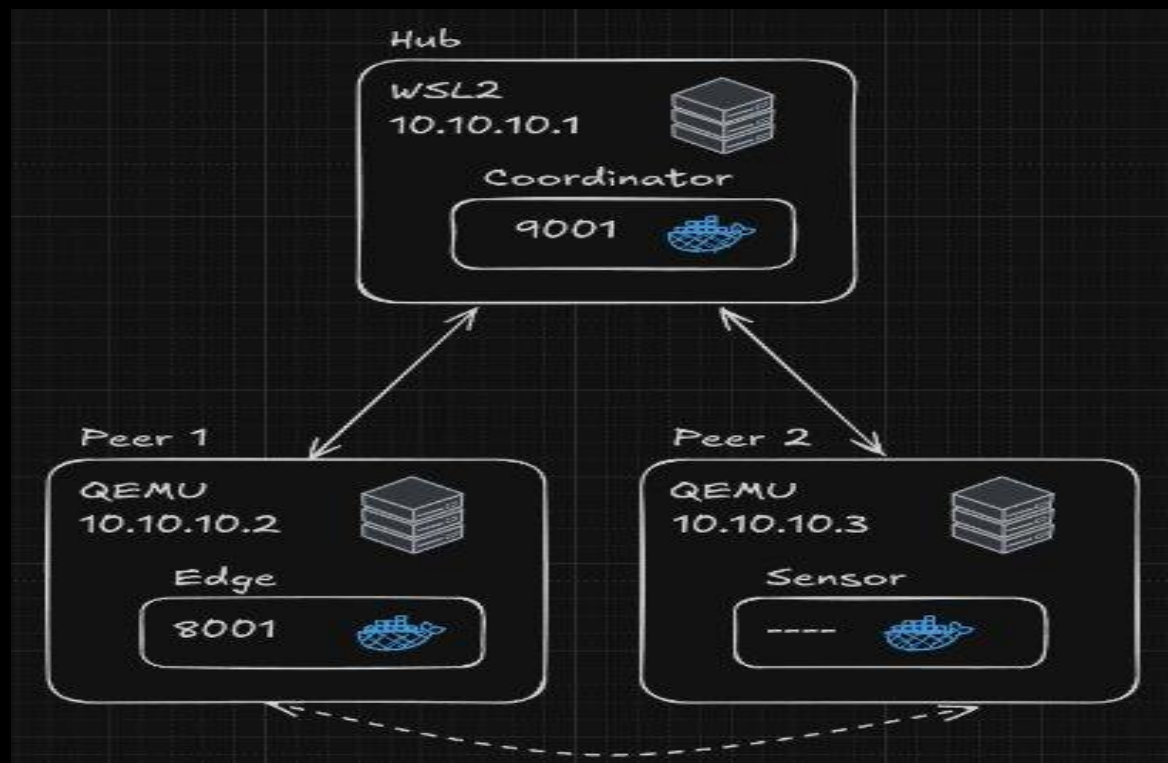
Reglas de Tráfico

Comando `tc netem` aplicado directamente sobre la interfaz virtual `wg0`.



Aislamiento de Red

Se evitó la aplicación de reglas sobre la red virtual interna del contenedor para mantener integridad.



CONTENEDORES DOCKER

Orquestación con Docker Compose

docker-compose.yml gestiona el despliegue coordinado de los tres servicios esenciales.



Servicio Coordinator

Expone el puerto 9001 para comunicación centralizada.



Servicio Edge

Opera en el puerto 8001 para el procesamiento en el borde.



Servicio Sensor

Envía lecturas de forma interna sin publicar puertos al host.



Optimización Runtime

Dockerfiles con compilación multi-stage para minimizar la imagen final.

→ docker git:(main) ✕ docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ee4683b3940b	docker-sensor	"/usr/local/bin/sens..."	24 hours ago	Up 9 seconds		docker-sensor-1
d6dad4f6d399	docker-edge	"/usr/local/bin/edge"	24 hours ago	Up 9 seconds	0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp	docker-edge-1
230101897da3	docker-coordinator	"/usr/local/bin/coor..."	24 hours ago	Up 9 seconds	0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp	docker-coordinator-1

→ docker git:(main) ✕ |

PROTOTIPO RUST

Crate shared: contratos JSON con serde

Estructuras: SensorReading, EdgeReport, Heartbeat



Sensor

Genera lecturas y las envía por TCP al servicio edge.



Edge

Recibe, procesa y reenvía los datos al coordinator.



Coordinator

Registra los datos finales y monitorea los heartbeats.

```

+ rust git:(main) X cargo run -p coordinator
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
  Running `target/debug/coordinator`
2026-05-05T02:28:40.581637Z INFO coordinator: COORDINATOR VIGILANDO (Datos y Heartbeats)
2026-05-05T02:28:41.668828Z INFO coordinator: Latido recibido tipo="HEARTBEAT" node=edge_angel ts=1777948121668
2026-05-05T02:28:44.669298Z INFO coordinator: Latido recibido tipo="HEARTBEAT" node=edge_angel ts=1777948124668
2026-05-05T02:28:47.669125Z INFO coordinator: Latido recibido tipo="HEARTBEAT" node=edge_angel ts=1777948127668
2026-05-05T02:28:48.771533Z INFO coordinator: Mensaje tipo="DATA" node=edge_angel avg=29.864561 anomaly=false latency_ms=0
2026-05-05T02:28:49.773108Z INFO coordinator: Mensaje tipo="DATA" node=edge_angel avg=33.482224 anomaly=false latency_ms=0
2026-05-05T02:28:50.669135Z INFO coordinator: Latido recibido tipo="HEARTBEAT" node=edge_angel ts=1777948130668
2026-05-05T02:28:50.774358Z INFO coordinator: Mensaje tipo="DATA" node=edge_angel avg=19.182476 anomaly=false latency_ms=0
2026-05-05T02:28:51.775812Z INFO coordinator: Mensaje tipo="DATA" node=edge_angel avg=27.273932 anomaly=false latency_ms=0
2026-05-05T02:28:52.777359Z INFO coordinator: Mensaje tipo="DATA" node=edge_angel avg=22.861595 anomaly=false latency_ms=0
2026-05-05T02:28:53.669039Z INFO coordinator: Latido recibido tipo="HEARTBEAT" node=edge_angel ts=1777948133668
2026-05-05T02:28:53.779197Z INFO coordinator: Mensaje tipo="DATA" node=edge_angel avg=30.140388 anomaly=false latency_ms=0
2026-05-05T02:28:54.780826Z INFO coordinator: Mensaje tipo="DATA" node=edge_angel avg=28.032087 anomaly=false latency_ms=0
2026-05-05T02:28:55.781523Z INFO coordinator: Mensaje tipo="DATA" node=edge_angel avg=27.620506 anomaly=false latency_ms=0
2026-05-05T02:28:56.669073Z INFO coordinator: Latido recibido tipo="HEARTBEAT" node=edge_angel ts=1777948136668

```

```

+ rust git:(main) X cargo run -p edge
  Finished `dev` profile [unoptimized + debuginfo] target
(s) in 0.01s
  Running `target/debug/edge`
>>> EDGE NODE ACTIVO (Enviando Heartbeats cada 3s)

```

```

+ rust git:(main) X cargo run -p sensor
  Finished `dev` profile [unoptimized + debuginfo] target(
s) in 0.01s
  Running `target/debug/sensor`
>>> SENSOR INICIADO (Intentando conectar al Edge...)
Enviado al Edge: 29.86 C
Enviado al Edge: 33.48 C
Enviado al Edge: 19.18 C
Enviado al Edge: 27.27 C
Enviado al Edge: 22.86 C
Enviado al Edge: 30.14 C
Enviado al Edge: 28.03 C
Enviado al Edge: 27.62 C
Enviado al Edge: 39.99 C
Enviado al Edge: 35.23 C
Enviado al Edge: 39.48 C
Enviado al Edge: 15.99 C
Enviado al Edge: 15.55 C
Enviado al Edge: 39.60 C

```


HEARTBEAT Y TOLERANCIA

Frecuencia de Latido

El nodo Edge envía un heartbeat automático cada 3 segundos para confirmar su estado.

Concurrencia con Tokio

Se ejecuta en una tarea asíncrona independiente utilizando `tokio::spawn`.

Discriminación de Tráfico

El Coordinator identifica y separa los mensajes de telemetría de los latidos de control.

Monitoreo de Estado

Proporciona visibilidad básica y en tiempo real del estado operativo de cada nodo Edge.

HEARTBEAT Y TOLERANCIA

```
→ rust git:(main) x cargo run -p coordinator
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
  Running `target/debug/coordinator`
2026-05-05T02:28:40.581637Z INFO coordinator: COORDINATOR VIGILANDO (Datos y Heartbeats)
2026-05-05T02:28:41.668828Z INFO coordinator: Latido recibido tipo="HEARTBEAT" node=edge_angel ts=1777948121668
2026-05-05T02:28:44.669298Z INFO coordinator: Latido recibido tipo="HEARTBEAT" node=edge_angel ts=1777948124668
2026-05-05T02:28:47.669125Z INFO coordinator: Latido recibido tipo="HEARTBEAT" node=edge_angel ts=1777948127668
```

DEMO TC NETEM



baseline.sh

Limpia reglas de red previas para un entorno controlado.



latencia_iot.sh

Aplica una emulación de retardo de red de $80\text{ms} \pm 20\text{ms}$ para tráfico IoT.

PASOS DE CONFIGURACIÓN DE RED



1. LIMPIAR REGLAS PREVIAS

```
$ ./baseline.sh
```



2. APLICAR LATENCIA IOT

```
$ ./latencia_iot.sh  
tc qdisc add dev wg0 \  
root netem delay 80ms 20ms
```



3. VERIFICAR

```
$ tc qdisc show dev wg0
```

METODOLOGÍA ÁGIL

Scrum ligero · Equipo de 2 · Sprint A (Avance) · Cierre: 2026-04-30

ID	TAREA	PRIORIDAD
A1	Verificar y documentar topología VPN WireGuard	P0
A2	Crear diagrama de topología actualizado	P1
A3	Configurar tc netem + evidencia iperf3	P0
A4	Crear docker-compose.yml con 3 contenedores	P0
B1	Crear workspace Cargo.toml con 3 crates	P0
B2	Definir estructuras de mensaje con serde	P1
B3	Implementar prototipo "hello distributed"	P0
B4	Implementar heartbeat básico	P1
B5	Plan de ejecución documentado para el PDF	P1

RIESGOS TÉCNICOS



Diseñar estrategia de reconexión controlada en Rust asíncrono para evitar race conditions



Sincronizar relojes entre nodos o definir medición basada en tiempo relativo para latencia E2E



Controlar crecimiento de buffers bajo red degradada y manejar backpressure



Verificar empaquetado del binario Rust con Docker multi-stage en todos los nodos



Definir secuencia de arranque tolerante a fallos para los tres servicios

RESUMEN GENERAL

BASE FUNCIONAL

LISTA PARA LA FASE FINAL



Pipeline sensor → edge → coordinator funcional



VPN y topología documentadas



Evidencia inicial con tc netem

RETOS Y ESTADO ACTUAL

RETOS ENCONTRADOS

- Aplicación controlada de `tc netem`
- Serialización **JSON** para contratos comunes
- Docker *multi-stage* (compilación vs ejecución)
- Heartbeat asíncrono sin bloqueo de flujo

ESTADO DEL SISTEMA

- VPN WireGuard operativa
- Pipeline sensor → edge → coordinator
- Heartbeat cada 3s funcional
- Escenario tc netem validado
- Base lista para fase final

POST-MORTEM TÉCNICO



Build Docker roto

Rust 1.76 incompatible con Cargo.lock. Se actualizó el builder a versión moderna.



Contrato desalineado

Flujo implementado sin validar contra documento. Se alinearon structs serde.



Bridge QEMU roto

Resuelto con hostfwd para recuperar conectividad sin depender del bridge.

QUÉ CAMBIARÍAMOS

- Fijar y validar toolchains Docker/Rust desde el día 1.
- Revisar el contrato del documento antes de cerrar la primera versión.
- Probar red de VMs al inicio y definir un plan B de conectividad.